question? Can we extend the idea of linear SVM to the nonlinear case? The key to constructing a nonlinear SVM is to observe that the observations in $\mathcal{L}$ only enter the dual optimization problem through the inner products $\langle \mathbf{x}_i, \mathbf{x}_j \rangle = \mathbf{x}_i^\tau \mathbf{x}_j$, $i, j = 1, 2, \ldots, n$.

11.3.1 Nonlinear Transformations

Suppose we transform each observation, $\mathbf{x}_i \in \Re^r$, in $\mathcal{L}$ using some nonlinear mapping $\mathbf{\Phi} : \Re^r \to \mathcal{H}$, where $\mathcal{H}$ is an $N_{\mathcal{H}}$-dimensional feature space. The nonlinear map $\mathbf{\Phi}$ is generally called the *feature map* and the space $\mathcal{H}$ is called the *feature space*. The space $\mathcal{H}$ may be very high-dimensional, possibly even infinite dimensional. We will generally assume that $\mathcal{H}$ is a Hilbert space of real-valued functions on $\Re$ with inner product $\langle \cdot, \cdot \rangle$ and norm $\| \cdot \|$.

Let

$$\mathbf{\Phi}(\mathbf{x}_i) = (\phi_1(\mathbf{x}_i), \cdots, \phi_{N_{\mathcal{H}}}(\mathbf{x}_i))^\tau \in \mathcal{H}, \quad i = 1, 2, \ldots, n. \tag{11.51}$$

The transformed sample is then $\{\mathbf{\Phi}(\mathbf{x}_i), y_i\}$, where $y_i \in \{-1, +1\}$ identifies the two classes. If we substitute $\mathbf{\Phi}(\mathbf{x}_i)$ for $\mathbf{x}_i$ in the development of the linear SVM, then data would only enter the optimization problem by way of the inner products $\langle \mathbf{\Phi}(\mathbf{x}_i), \mathbf{\Phi}(\mathbf{x}_j) \rangle = \mathbf{\Phi}(\mathbf{x}_i)^\tau \mathbf{\Phi}(\mathbf{x}_j)$. The difficulty in using nonlinear transformations in this way is computing such inner products in high-dimensional space $\mathcal{H}$.

11.3.2 The "Kernel Trick"

The idea behind nonlinear SVM is to find an optimal separating hyperplane (with or without slack variables, as appropriate) in high-dimensional feature space $\mathcal{H}$ just as we did for the linear SVM in input space. Of course, we would expect the dimensionality of $\mathcal{H}$ to be a huge impediment to constructing an optimal separating hyperplane (and classification rule) because of the curse of dimensionality. The fact that this does not become a problem in practice is due to the "kernel trick," which was first applied to SVMs by Cortes and Vapnik (1995).

The so-called kernel trick is a wonderful idea that is widely used in algorithms for computing inner products of the form $\langle \Phi(\mathbf{x}_i), \Phi(\mathbf{x}_j) \rangle$ in feature space $\mathcal{H}$. The trick is that instead of computing these inner products in $\mathcal{H}$, which would be computationally expensive because of its high dimensionality, we compute them using a nonlinear kernel function, $K(\mathbf{x}_i, \mathbf{x}_j) = \langle \Phi(\mathbf{x}_i), \Phi(\mathbf{x}_j) \rangle$, in input space, which helps speed up the computations. Then, we just compute a *linear* SVM, but where the computations are carried out in some other space.

11.3.3 Kernels and Their Properties

A *kernel* K is a function $K : \Re^r \times \Re^r \to \Re$ such that, for all $\mathbf{x}, \mathbf{y} \in \Re^r$,

$$K(\mathbf{x}, \mathbf{y}) = \langle \Phi(\mathbf{x}), \Phi(\mathbf{y}) \rangle. \tag{11.52}$$

The kernel function is designed to compute inner-products in $\mathcal{H}$ by using only the original input data. Thus, wherever we see the inner product $\langle \Phi(\mathbf{x}), \Phi(\mathbf{y}) \rangle$, we substitute the kernel function $K(\mathbf{x}, \mathbf{y})$. The choice of K implicitly determines both Φ and $\mathcal{H}$. The big advantage to using kernels as inner products is that if we are given a kernel function K, then we do not need to know the explicit form of Φ.

We require that the kernel function be symmetric, $K(\mathbf{x}, \mathbf{y}) = K(\mathbf{y}, \mathbf{x})$, and satisfy an inequality, $[K(\mathbf{x}, \mathbf{y})]^2 \leq K(\mathbf{x}, \mathbf{x})K(\mathbf{y}, \mathbf{y})$. derived from the Cauchy–Schwarz inequality. If $K(\mathbf{x}, \mathbf{x}) = 1$ for all $\mathbf{x} \in \Re^r$, this implies that $\|\Phi(\mathbf{x})\|_{\mathcal{H}} = 1$. A kernel K is said to have the *reproducing property* if, for any $f \in \mathcal{H}$,

$$\langle f(\cdot), K(\mathbf{x}, \cdot) \rangle = f(\mathbf{x}). \tag{11.53}$$

If K has this property, we say it is a *reproducing kernel*. K is also called the *representer of evaluation*. In particular, if $f(\cdot) = K(\cdot, \mathbf{x})$, then,

$$\langle K(\mathbf{x}, \cdot), K(\mathbf{y}, \cdot) \rangle = K(\mathbf{x}, \mathbf{y}). \tag{11.54}$$

Let $\mathbf{x}_1, \ldots, \mathbf{x}_n$ be any set of n points in $\mathcal{R}^r$. Then, the $(n \times n)$-matrix $\mathbf{K} = (K_{ij})$, where $K_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$, $i, j = 1, 2, \ldots, n$, is called the *Gram* (or *kernel*) matrix of K with respect to $\mathbf{x}_1, \ldots, \mathbf{x}_n$. If the Gram matrix $\mathbf{K}$ satisfies $\mathbf{u}^\tau \mathbf{K} \mathbf{u} \geq 0$, for any n-vector $\mathbf{u}$, then it is said to be *nonnegative-definite* with nonnegative eigenvalues, in which case we say that K is a nonnegative-definite kernel[1] (or *Mercer kernel*).

If K is a specific Mercer kernel on $\mathcal{R}^r \times \mathcal{R}^r$, we can always construct a unique Hilbert space $\mathcal{H}_K$, say, of real-valued functions for which K is its reproducing kernel. We call $\mathcal{H}_K$ a (real) *reproducing kernel Hilbert space* (rkhs). We write the inner-product and norm of $\mathcal{H}_K$ by $\langle \cdot, \cdot \rangle_{\mathcal{H}_K}$ (or just $\langle \cdot, \cdot \rangle$ when K is understood) and $\| \cdot \|_{\mathcal{H}_K}$, respectively.

11.3.4 Examples of Kernels

An example of a kernel is the *inhomogeneous polynomial kernel of degree d*,

$$K(\mathbf{x}, \mathbf{y}) = (\langle \mathbf{x}, \mathbf{y} \rangle + c)^d, \quad \mathbf{x}, \mathbf{y} \in \Re^r, \tag{11.55}$$

[1] In the machine-learning literature, nonnegative-definite matrices and kernels are usually referred to as positive-definite matrices and kernels, respectively.

TABLE 11.1. *Kernel functions, $K(\mathbf{x}, \mathbf{y})$, where $\sigma > 0$ is a scale parameter, $a, b, c \geq 0$, and d is an integer. The Euclidean norm is $\|\mathbf{x}\|^2 = \mathbf{x}^\tau \mathbf{x}$.*

Kernel	$K(\mathbf{x}, \mathbf{y})$
Polynomial of degree d	$(\langle \mathbf{x}, \mathbf{y} \rangle + c)^d$
Gaussian radial basis function	$\exp\left\{-\frac{\|\mathbf{x}-\mathbf{y}\|^2}{2\sigma^2}\right\}$
Laplacian	$\exp\left\{-\frac{\|\mathbf{x}-\mathbf{y}\|}{\sigma}\right\}$
Thin-plate spline	$\left(\frac{\|\mathbf{x}-\mathbf{y}\|}{\sigma}\right)^2 \log_e\left\{\frac{\|\mathbf{x}-\mathbf{y}\|}{\sigma}\right\}$
Sigmoid	$\tanh(a\langle \mathbf{x}, \mathbf{y} \rangle + b)$

where c and d are parameters. The homogeneous form of the kernel occurs when $c = 0$ in (12.55). If $d = 1$ and $c = 0$, the feature map reduces to the identity. Usually, we take $c > 0$. A simple nonlinear map is given by the case $r = 2$ and $d = 2$. If $\mathbf{x} = (x_1, x_2)^\tau$ and $\mathbf{y} = (y_1, y_2)^\tau$, then,

$$K(\mathbf{x}, \mathbf{y}) = (\langle \mathbf{x}, \mathbf{y} \rangle + c)^2 = (x_1 y_1 + x_2 y_2 + c)^2 = \langle \Phi(\mathbf{x}), \Phi(\mathbf{y}) \rangle,$$

where $\Phi(\mathbf{x}) = (x_1^2, x_2^2, \sqrt{2}x_1 x_2, \sqrt{2c}x_1, \sqrt{2}x_2, c)^\tau$ and similarly for $\Phi(\mathbf{y})$. In this example, the function $\Phi(\mathbf{x})$ consists of six features ($\mathcal{H} = \Re^6$), all monomials having degree at most 2. For this kernel, we see that c controls the magnitudes of the constant term and the first-degree term.

In general, there will be $\dim(\mathcal{H}) = \binom{r+d}{d}$ different features, consisting of all monomials having degree at most d. The dimensionality of $\mathcal{H}$ can rapidly become very large: for example, in visual recognition problems, data may consist of 16×16 pixel images (so that each image is turned into a vector of dimension $r = 256$); if $d = 2$, then $\dim(\mathcal{H}) = 33,670$, whereas if $d = 4$, we have $\dim(\mathcal{H}) = 186,043,585$.

Other popular kernels, such as the *Gaussian radial basis function (RBF)*, the *Laplacian kernel*, the *thin-plate spline kernel*, and the *sigmoid kernel*, are given in Table 11.1. Strictly speaking, the sigmoid kernel is not a kernel (it satisfies Mercer's conditions only for certain values of a and b), but it has become very popular in that role in certain situations (e.g., two-layer neural networks).

The Gaussian RBF, Laplacian, and thin-plate spline kernels are examples of *translation-invariant* (or *stationary*) *kernels* having the general form

$K(\mathbf{x}, \mathbf{y}) = k(\mathbf{x} - \mathbf{y})$, where $k : \Re^r \to \Re$. The polynomial kernel is an example of a nonstationary kernel. A stationary kernel $K(\mathbf{x}, \mathbf{y})$ is *isotropic* if it depends only upon the distance $\delta = \|\mathbf{x} - \mathbf{y}\|$, i.e., if $K(\mathbf{x}, \mathbf{y}) = k(\delta)$, scaled to have $k(0) = 1$.

It is not always obvious which kernel to choose in any given application. Prior knowledge or a search through the literature can be helpful. If no such information is available, the best approach is to try either a Gaussian RBF, which has only a single parameter (σ) to be determined, or a polynomial kernel of low degree ($d = 1$ or 2). If necessary, more complicated kernels can then be applied to compare results.

String Kernels for Text Categorization

Text categorization is the assignment of natural-language text (or hypertext) documents into a given number of predefined categories based upon the content of those documents (see Section 2.2.1). Although manual categorization of text documents is currently the norm (e.g., using folders to save files, e-mail messages, URLs, etc.), some text categorization is automated (e.g., filters for spam or junk mail to help users cope with the sheer volume of daily e-mail messages). To reduce costs of text categorization tasks, we should expect a greater degree of automation to be present in the future.

In text-categorization problems, *string kernels* have been proposed based upon ideas derived from bioinformatics (see, e.g., Lodhi, Saunders, Shawe-Taylor,Cristianini, and Watkins, 2002).

Let $\mathcal{A}$ be a finite alphabet. A "string"

$$s = s_1 s_2 \cdots s_{|s|} \tag{11.56}$$

is a finite sequence of elements of $\mathcal{A}$, including the empty sequence, where $|s|$ denotes the length of s. We call u a *subsequence* of s (written $u = s(\mathbf{i})$) if there are indices $\mathbf{i} = (i_1, i_2, \cdots, i_{|u|})$, with $1 \leq i_1 < \cdots < i_{|u|} \leq |s|$, such that $u_j = s_{i_j}$, $j = 1, 2, \ldots, |u|$. If the indices $\mathbf{i}$ are contiguous, we say that u is a *substring* of s. The length of u in s is

$$\ell(\mathbf{i}) = i_{|u|} - i_1 + 1, \tag{11.57}$$

which is the number of elements of s overlaid by the subsequence u. For example, let s be the string "cat" ($s_1 = c, s_2 = a, s_3 = t$, $|s| = 3$), and consider all possible 2-symbol sequences, "ca," "ct," and "at," derived from s. For the string $u =$ ca, we have that $u_1 = \mathrm{c} = s_1, u_2 = \mathrm{a} = s_2$, whence, $u = s(\mathbf{i})$, where $\mathbf{i} = (i_1, i_2) = (1, 2)$. Thus, $\ell(\mathbf{i}) = 2$. Similarly, for the subsequence $u =$ ct, $u_1 = \mathrm{c} = s_1, u_2 = \mathrm{t} = s_3$, whence, $\mathbf{i} = (i_1, i_2) = (1, 3)$, and $\ell(\mathbf{i}) = 3$. Also, the subsequence $u =$ at has $u_1 = \mathrm{a} = s_2, u_2 = \mathrm{t} = s_3$, whence, $\mathbf{i} = (2, 3)$, and $\ell(\mathbf{i}) = 2$.

If $D = \mathcal{A}^m$ is the set of all finite strings of length at most m from $\mathcal{A}$, then, the feature space for a string kernel is $\Re^D$. The feature map Φ_u, operating on a string $s \in \mathcal{A}^m$, is characterized in terms of a given string $u \in \mathcal{A}^m$. To deal with noncontiguous subsequences, define $\lambda \in (0, 1)$ as the *drop-off rate* (or *decay factor*); we use λ to weight the interior gaps in the subsequences. The degree of importance we put into a contiguous subsequence is reflected in how small we take the value of λ. The value $\Phi_u(s)$ is computed as follows: identify all subsequences (indexed by $\mathbf{i}$) of s that are identical to u; for each such subsequence, raise λ to the power $\ell(\mathbf{i})$; and then sum the results over all subsequences. Because $\lambda < 1$, larger values of $\ell(\mathbf{i})$ carry less weight than smaller values of $\ell(\mathbf{i})$. We write

$$\Phi_u(s) = \sum_{\mathbf{i}:u=s(\mathbf{i})} \lambda^{\ell(\mathbf{i})}, \quad u \in \mathcal{A}^m. \tag{11.58}$$

In our example above, $\Phi_{\text{ca}}(\text{cat}) = \lambda^2$, $\Phi_{\text{ct}}(\text{cat}) = \lambda^3$, and $\Phi_{\text{at}}(\text{cat}) = \lambda^2$.

Two documents are considered to be "similar" if they have many subsequences in common: the more subsequences they have in common, the more similar they are deemed to be. Note that the degree of contiguity present in a subsequence determines the weight of that substring in the comparison; the closer the subsequence is to a contiguous substring, the more it should contribute to the comparison.

Let s and t be two strings. The kernel associated with the feature maps corresponding to s and t is given by the sum of inner products for all *common* substrings of length m,

$$\begin{aligned} K_m(s,t) &= \sum_{u\in\mathcal{D}} \langle \Phi_u(s), \Phi_u(t) \rangle \\ &= \sum_{u\in\mathcal{D}} \sum_{\mathbf{i}:u=s(\mathbf{i})} \sum_{\mathbf{j}:u=s(\mathbf{j})} \lambda^{\ell(\mathbf{i})+\ell(\mathbf{j})}. \end{aligned} \tag{11.59}$$

The kernel (11.59) is called a *string kernel* (or a *gap-weighted subsequences kernel*). For the example, let t be the string "car" ($t_1 = \text{c}, t_2 = \text{a}, t_3 = \text{r}$, $|t| = 3$). Note that the strings "cat" and "car" are both substrings of the string "cart." The three 2-symbol substrings of t are "ca," "cr," and "ar." For these substrings, we have that $\Phi_{\text{ca}}(\text{car}) = \lambda^2, \Phi_{\text{cr}}(\text{car}) = \lambda^3$, and $\Phi_{\text{ar}}(\text{car}) = \lambda^2$. The inner product (11.62) is given by $K_2(\text{cat}, \text{car}) = \langle \Phi_{\text{ca}}(\text{cat}), \Phi_{\text{ca}}(\text{car}) \rangle = \lambda^4$.

The feature maps in feature space are usually normalized to remove any bias introduced by document length. This is equivalent to normalizing the kernel (11.59),

$$K_m^*(s,t) = \frac{K_m(s,t)}{\sqrt{K_m(s,s)K_m(t,t)}}. \tag{11.60}$$

For our example, $K_2(\text{cat},\text{cat}) = \langle\Phi_{\text{ca}}(\text{cat}),\Phi_{\text{ca}}(\text{cat})\rangle+\langle\Phi_{\text{ct}}(\text{cat}),\Phi_{\text{ct}}(\text{cat})\rangle+\langle\Phi_{\text{at}}(\text{cat}),\Phi_{\text{at}}(\text{cat})\rangle = \lambda^6 + 2\lambda^4$, and, similarly, $K_2(\text{car},\text{car}) = \lambda^6 + 2\lambda^4$, whence, $K_2^*(\text{cat},\text{car}) = \lambda^4/(\lambda^6 + 2\lambda^4) = 1/(\lambda^2 + 2)$.

The parameters of the string kernel (11.59) are m and λ. The choices of $m = 5$ and $\lambda = 0.5$ have been found to perform well on segments of certain data sets (e.g., on subsets of the Reuters-21578 data) but do not fare as well when applied to the full data set.

11.3.5 Optimizing in Feature Space

Let K be a kernel. Suppose, first, that the observations in $\mathcal{L}$ are linearly separable in the feature space corresponding to the kernel K. Then, the dual optimization problem is to find $\boldsymbol{\alpha}$ and β_0 to

$$\textit{maximize} \quad F_D(\boldsymbol{\alpha}) = \mathbf{1}_n^\tau\boldsymbol{\alpha} - \frac{1}{2}\boldsymbol{\alpha}^\tau\mathbf{H}\boldsymbol{\alpha} \tag{11.61}$$

$$\textit{subject to} \quad \boldsymbol{\alpha} \geq \mathbf{0},\ \boldsymbol{\alpha}^\tau\mathbf{y} = 0, \tag{11.62}$$

where $\mathbf{y} = (y_1,\cdots,y_n)^\tau$, $\mathbf{H} = (H_{ij})$, and

$$H_{ij} = y_iy_jK(\mathbf{x}_i,\mathbf{x}_j) = y_iy_jK_{ij}, \quad i,j = 1,2,\ldots,n. \tag{11.63}$$

Because K is a kernel, the Gram matrix $\mathbf{K} = (K_{ij})$ is nonnegative-definite, and so is the matrix $\mathbf{H}$ with elements (11.63). Hence, the functional $F_D(\boldsymbol{\alpha})$ is convex (see Exercise 11.8). So, there is a unique solution to this constrained optimization problem. If $\widehat{\boldsymbol{\alpha}}$ and $\widehat{\beta}_0$ solve this problem, then, the SVM decision rule is $\text{sign}\{\widehat{f}(\mathbf{x})\}$, where

$$\widehat{f}(\mathbf{x}) = \widehat{\beta}_0 + \sum_{i\in sv}\widehat{\alpha}_iy_iK(\mathbf{x},\mathbf{x}_i) \tag{11.64}$$

is the optimal separating hyperplane in the feature space corresponding to the kernel K.

In the nonseparable case, using the kernel K, the dual problem of the 1-norm soft-margin optimization problem is to find $\boldsymbol{\alpha}$ to

$$\textit{maximize} \quad F_D^*(\boldsymbol{\alpha}) = \mathbf{1}_n^\tau\boldsymbol{\alpha} - \frac{1}{2}\boldsymbol{\alpha}^\tau\mathbf{H}\boldsymbol{\alpha} \tag{11.65}$$

$$\textit{subject to} \quad \mathbf{0} \leq \boldsymbol{\alpha} \leq C\mathbf{1}_n,\ \boldsymbol{\alpha}^\tau\mathbf{y} = 0, \tag{11.66}$$

where $\mathbf{y}$ and $\mathbf{H}$ are as above. For an optimal solution, the Karush–Kuhn–Tucker conditions, (11.42)–(11.47), must hold for the primal problem. So, a solution, $\boldsymbol{\alpha}$, to this problem has to satisfy all those conditions. Fortunately, it suffices to check a simpler set of conditions: we have to check that $\boldsymbol{\alpha}$

satisfies (11.66) and that (11.42) holds for all points where $0 \leq \alpha_i < C$ and $\xi_i = 0$, and also for all points where $\alpha_i = C$ and $\xi_i \geq 0$.

11.3.6 Grid Search for Parameters

We need to determine two parameters when using a Gaussian RBF kernel, namely, the cost, C, of violating the constraints and the kernel parameter $\gamma = 1/\sigma^2$. The parameter C in the box constraint can be chosen by searching a wide range of values of C using either CV (usually, 10-fold) on $\mathcal{L}$ or an independent validation set of observations. In practice, it is usual to start the search by trying several different values of C, such as 10, 100, 1,000, 10,000, and so on. A initial grid of values of γ can be selected by trying out a crude set of possible values, say, 0.00001, 0.0001, 0.001, 0.01, 0.1, and 1.0.

When there appears to be a minimum CV misclassification rate within an interval of the two-way grid, we make the grid search finer within that interval. Armed with a two-way grid of values of (C, γ), we apply CV to estimate the generalization error for each cell in that grid. The (C, γ) that has the smallest CV misclassification rate is selected as the solution to the SVM classification problem.

11.3.7 Example: E-mail or Spam?

This example (`spambase`) was described in Section 8.4, where we applied LDA and QDA to a collection of 4,601 messages, comprising 1,813 spam e-mails and 2,788 non-spam e-mails. There are 57 variables (attributes) and each message is labeled as one of the two classes `email` or `spam`.

Here we apply nonlinear SVM (R package `libsvm`) using a Gaussian RBF kernel to the 4,601 messages. The SVM solution depends upon the cost C of violating the constraints and the variance, σ^2, of the Gaussian RBF kernel. After applying a trial-and-error method, we used the following grid of values for C amd $\gamma = 1/\sigma^2$:

$C = 10, 80, 100, 200, 500, 1{,}000,$

$\gamma = 0.00001(0.00001)0.0001(0.0001)0.002(0.001)0.01(0.01)0.04.$

In Figure 11.3, we plot the values of the 10-fold CV misclassification rate against the values of γ listed above, where each curve (connected set of points) represents a different value of C. For each C, we see that the CV/10 misclassification curves have similar shapes: a minimum value for γ very close to zero, and for values of γ away from zero, the curve trends upwards. In this initial search, we find a minimum CV/10 misclassification rate of 8.06% at $(C, \gamma) = (500, 0.0002)$ and (1,000, 0.0002). We see that the general

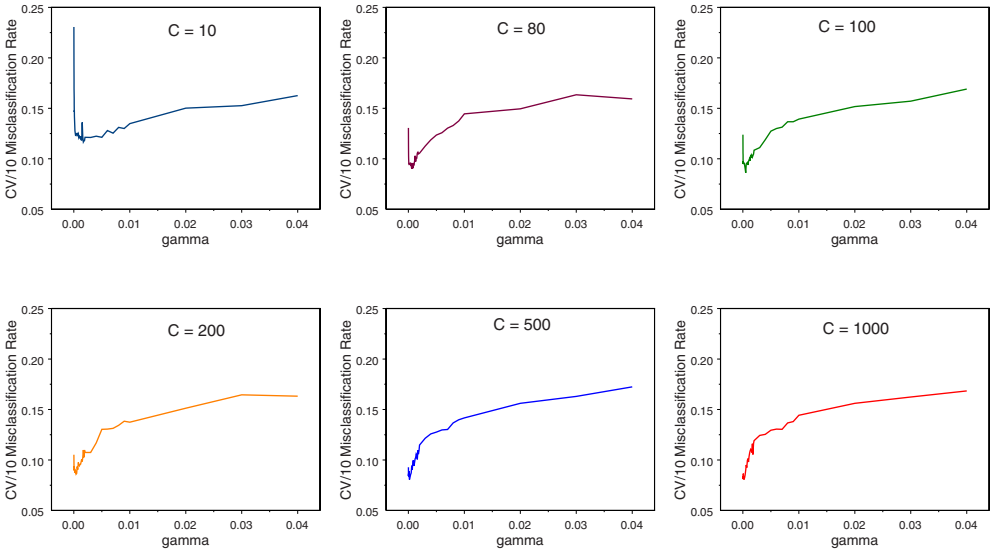

FIGURE 11.3. *SVM cross-validation misclassification rate curves for the spambase data. Initial grid search for the minimum 10-fold CV misclassification rate using* $0.00001 \leq \gamma \leq 0.04$. *The curves correspond to* $C = 10$ *(dark blue), 80 (brown), 100 (green), 200 (orange), 500 (light blue), and 1,000 (red). Within this intial grid search, the minimum CV/10 misclassification rate is 8.06%, which occurs at* $(C, \gamma) = (500, 0.0002)$ *and (1,000, 0.0002).*

level of the misclassification rate tends to decrease as C increases and γ decreases together.

A detailed investigation of $C > 1000$ and γ close to zero reveals a minimum CV/10 misclassification rate of 6.91% at $C = 11,000$ and $\gamma = 0.00001$, corresponding to the following 10 CV estimates of the true classification rate:

0.9043, 0.9478, 0.9304, 0.9261, 0.9109,

0.9413, 0.9326, 0.9500. 0.9326, 0.9328.

This solution has 931 support vectors (482 e-mails, 449 spam), which means that a large percentage (79.8%) of the messages (82.7% of the e-mails and 75.2% of the spam) are not support points. Of the 4,601 messages, 2,697 e-mails and 1,676 spam are correctly classified (228 misclassified), yielding an apparent error rate of 4.96%.

This example turns out to be more computationally intensive than are the other binary-classification examples discussed in this chapter. Although the value of γ has very little effect on the speed of computating the 10-fold CV error rate, the speed of computation does depend upon C: as we increase the value of C, the speed of computation slows down considerably.

TABLE 11.2. *Summary of support vector machine (SVM) application to data sets for binary classification. Listed are the sample size (n), number of variables (r), and number of classes (K). Also listed for each data set is the 10-fold cross-validation (CV/10) misclassification rates corresponding to the best choice of (C, γ) for the SVM. The data sets are listed in increasing order of LDA misclassification rates (see Table 8.5).*

Data Set	n	r	K	SVM–CV/10
Breast cancer (logs)	569	30	2	0.0158
Spambase	4601	57	2	0.0691
Ionosphere	351	33	2	0.0427
Sonar	208	60	2	0.1010
BUPA liver disorders	345	6	2	0.2522

Also worth noting is that for fixed γ, increasing C reduces the number of support vectors and the apparent error rate. We cannot make similar general statements about fixed C and increasing γ; however, for fixed C, we generally see that the number of support vectors tends to increase (but not always) with increasing γ.

The nonlinear SVM is clearly a better classifier for this example than is LDA or QDA, whose leave-one-out CV misclassification rate is around 11% for LDA and 17% for QDA, but the amount of computational work involved in the grid search for the SVM solution is much greater and, hence, a lot more expensive.

11.3.8 Binary Classification Examples

We apply the SVM algorithm to the binary classification examples of Section 8.4: the log-transformed breast cancer data, the ionosphere data, the BUPA liver disorders data, the sonar data, and the spambase data. Except for spambase, computations for these examples were very fast.

In Table 11.2, we list the minimum 10-fold CV misclassification rate for each data set. Comparing these results to those of LDA (see Table 8.5, where we used leave-one-out CV), we see that SVM produces remarkable decreases in misclassification rates: the breast cancer rate decreased from 11.3% to 1.58%, the spambase rate decreased from 11.3% to 6.91%, the ionosphere rate decreased from 13.7% to 4.27%, the sonar rate decreased from 24.5% to 10.1%, and the BUPA liver disorders rate decreased from 30.1% to 25.22%.

11.3.9 SVM as a Regularization Method

The SVM classifier can also be regarded as the solution to a particular regularization problem. Let $f \in \mathcal{H}_K$, the reproducing kernel Hilbert space

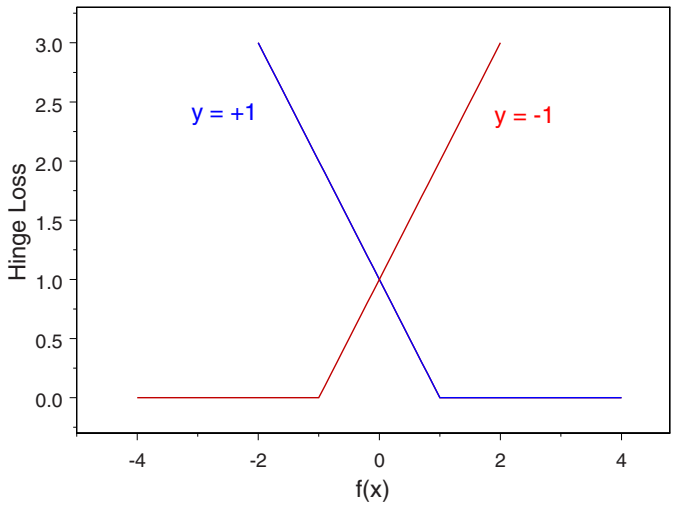

FIGURE 11.4. *Hinge loss function* $(1-yf(\mathbf{x}))_+$ *for* $y=-1$ *and* $y=+1$.

(rkhs) associated with the kernel K, with $\| f \|_{\mathcal{H}_K}{}^2$ the squared-norm of f in $\mathcal{H}_K$.

Consider the classification error, $y_i - f(\mathbf{x}_i)$, where $y_i \in \{-1, +1\}$. Then,

$$|y_i - f(\mathbf{x}_i)| = |y_i(1 - y_i f(\mathbf{x}_i))| = |1 - y_i f(\mathbf{x}_i)| = (1 - y_i f(\mathbf{x}_i))_+, \quad (11.67)$$

$i = 1, 2, \ldots, n$, where $(x)_+ = \max\{x, 0\}$. The quantity $(1 - y_i f(\mathbf{x}_i))_+$, which could be zero if all $\mathbf{x}_i$ are correctly classified, is called the *hinge loss function* and is displayed in Figure 11.4. The hinge loss plays a vital role in SVM methodology; indeed, it has been shown to be Bayes consistent for classification in the sense that minimizing the loss function yields the Bayes rule (Lin, 2002). The hinge loss is also related to the misclassification loss function $I_{[y_i C(\mathbf{x}_i) \leq 0]} = I_{[y_i f(\mathbf{x}_i) \leq 0]}$. When $f(\mathbf{x}_i) = \pm 1$, the hinge loss is twice the misclassification loss; otherwise, the ratio of the two losses depends upon the sign of $y_i f(\mathbf{x}_i)$.

We wish to find a function $f \in \mathcal{H}_K$ to minimize a penalized version of the hinge loss. Specifically, we wish to find $f \in \mathcal{H}_K$ to

$$\textit{minimize} \quad \frac{1}{n}\sum_{i=1}^{n}(1 - y_i f(\mathbf{x}_i))_+ + \lambda \| f \|_{\mathcal{H}_K}{}^2, \quad (11.68)$$

where $\lambda > 0$. In (11.69), the first term, $n^{-1}\sum_{i=1}^{n}(1 - y_i f(\mathbf{x}_i))_+$, measures the distance of the data from separability, and the second term, $\lambda \| f \|^2_{\mathcal{H}_K}$, penalizes overfitting. The tuning parameter λ balances the trade-off between estimating f (the first term) and how well f can be approximated

(the second term). After the minimizing f has been found, the SVM classifier is $C(\mathbf{x}) = \text{sign}\{f(\mathbf{x})\}$, $\mathbf{x} \in \mathcal{R}^r$.

The optimizing criterion (11.68) is nondifferentiable due to the shape of the hinge-loss function. Fortunately, we can rewrite the problem in a slightly different form and thereby solve it.

We start from the fact that every $f \in \mathcal{H}$ can be written uniquely as the sum of two terms:

$$f(\cdot) = f^{\parallel}(\cdot) + f^{\perp}(\cdot) = \sum_{i=1}^{n} \alpha_i K(\mathbf{x}_i, \cdot) + f^{\perp}(\cdot), \tag{11.69}$$

where $f^{\parallel} \in \mathcal{H}_K$ is the projection of f onto the subspace $\mathcal{H}_K$ of $\mathcal{H}$ and $f^{\perp}$ is in the subspace perpendicular to $\mathcal{H}_K$; that is, $\langle f^{\perp}(\cdot), K(\mathbf{x}_i, \cdot)\rangle_{\mathcal{H}} = 0$, $i = 1, 2, \ldots, n$. We can write $f(\mathbf{x}_i)$ via the reproducing property as follows:

$$f(\mathbf{x}_i) = \langle f(\cdot), K(\mathbf{x}_i, \cdot)\rangle = \langle f^{\parallel}(\cdot), K(\mathbf{x}_i, \cdot)\rangle + \langle f^{\perp}(\cdot), K(\mathbf{x}_i, \cdot)\rangle. \tag{11.70}$$

Because the second term on the rhs is zero, then,

$$f(\mathbf{x}) = \sum_{i=1}^{n} \alpha_i K(\mathbf{x}_i, \mathbf{x}), \tag{11.71}$$

independent of $f^{\perp}$, where we used (11.69) and $\langle K(\mathbf{x}_i, \cdot), K(\mathbf{x}_j, \cdot)\rangle_{\mathcal{H}_K} = K(\mathbf{x}_i, \mathbf{x}_j)$. Now, from (11.69),

$$\begin{aligned} \| f \|^2_{\mathcal{H}_K} &= \| \sum_i \alpha_i K(\mathbf{x}_i, \cdot) + f^{\perp} \|^2_{\mathcal{H}_K} \\ &= \| \sum_i \alpha_i K(\mathbf{x}_i, \cdot) \|^2_{\mathcal{H}_K} + \| f^{\perp} \|^2_{\mathcal{H}_K} \\ &\geq \| \sum_i \alpha_i K(\mathbf{x}_i, \cdot) \|^2_{\mathcal{H}_K}, \end{aligned} \tag{11.72}$$

with equality iff $f^{\perp} = 0$, in which case any $f \in \mathcal{H}_K$ that minimizes (11.68) admits a representation of the form (11.71). This important result is known as the *representer theorem* (Kimeldorf and Wahba, 1971); it says that the minimizing f (which would live in an infinite-dimensional rkhs if, for example, the kernel is a Gaussian RBF) can be written as a linear combination of a reproducing kernel evaluated at each of the n data points.

From (11.72), we have that $\| f \|^2_{\mathcal{H}_K} = \sum_i \sum_j \alpha_i \alpha_j K(\mathbf{x}_i, \mathbf{x}_j) = \| \boldsymbol{\beta} \|^2$, where $\boldsymbol{\beta} = \sum_{i=1}^{n} \alpha_i \boldsymbol{\Phi}(\mathbf{x}_i)$. If the space $\mathcal{H}_K$ consists of linear functions of the form $f(\mathbf{x}) = \beta_0 + \boldsymbol{\Phi}(\mathbf{x})^{\tau}\boldsymbol{\beta}$ with $\| f \|^2_{\mathcal{H}_K} = \| \boldsymbol{\beta} \|^2$, then the problem of finding f in (11.68) is equivalent to one of finding β_0 and $\boldsymbol{\beta}$ to

$$\textit{minimize} \quad \frac{1}{n} \sum_{i=1}^{n} (1 - y_i(\beta_0 + \boldsymbol{\Phi}(\mathbf{x}_i)^{\tau}\boldsymbol{\beta}))_+ + \lambda \| \boldsymbol{\beta} \|^2 . \tag{11.73}$$